

Assignment: Applying the Decorator Pattern in Software Design

Course: CP353002 Principles of software Design and Development

Topic: Structural Design Patterns - Decorator Pattern

Total Score: 100 Points

Learning Outcomes

เมื่อจบงานนี้ นักศึกษาจะสามารถ

1. วิเคราะห์ปัญหาที่เหมาะสมกับ Decorator Pattern
2. ออกแบบ UML Class Diagram สำหรับ Decorator Pattern
3. พัฒนาระบบโดยใช้หลัก Composition over Inheritance
4. อธิบายความสัมพันธ์ระหว่าง Decorator Pattern และ Open-Closed Principle
5. เปรียบเทียบ Decorator Pattern กับการใช้ Inheritance แบบดั้งเดิม

Case Study

บริษัทพัฒนาซอฟต์แวร์ได้รับมอบหมายให้ออกแบบระบบสั่งกาแฟสำหรับร้านกาแฟแห่งหนึ่ง

ลูกค้าสามารถเลือกเครื่องดื่มหลักและเพิ่มส่วนประกอบเสริมได้อย่างอิสระ

Base Products

- Espresso (\$2.00)
- Americano (\$2.50)

Optional Features

- Milk (+\$0.50)
- Sugar (+\$0.20)
- Whipped Cream (+\$0.70)

ในอนาคตอาจมี Add-on ใหม่เพิ่มเติม เช่น

- Caramel
- Chocolate
- Honey
- Ice Cream

ดังนั้นระบบต้องสามารถขยายความสามารถได้โดยไม่ต้องแก้ไขคลาสเดิม

Tasks

Part 1: Problem Analysis (15 คะแนน)

จงตอบคำถามต่อไปนี้

1.1 หากใช้ Inheritance เพียงอย่างเดียว จะเกิดปัญหาอะไรขึ้นเมื่อมีการเพิ่ม Add-on จำนวนมาก

ตอบ จะเกิดปัญหา Class Explosion เนื่องจากต้องสร้างคลาสใหม่สำหรับกาแฟแต่ละชนิดและทุก combination ของ add-on เมื่อ add-on เพิ่มขึ้นจำนวนคลาสที่ต้องสร้างและดูแลก็จะเพิ่มขึ้นอย่างรวดเร็ว ทำให้โครงสร้างระบบซับซ้อน ดูแลรักษายาก และไม่ยืดหยุ่นต่อการเพิ่มฟังก์ชันใหม่

1.2 เพราะเหตุใด Decorator Pattern จึงเหมาะสมกับระบบนี้

ตอบ เพราะเนื่องจากลูกค้าสามารถเลือก base coffee และเพิ่ม Add-on ได้อย่างอิสระและสามารถเพิ่ม Add-on ได้หลายชั้น Decorator ช่วยให้สามารถนำ Add-on แต่ละชนิดมาห่อ Object เดิมเพื่อเพิ่มฟังก์ชันและราคา โดยไม่จำเป็นต้องสร้างคลาสใหม่สำหรับทุก combination ของกาแฟและ Add-on นอกจากนี้ยังสามารถเพิ่ม Decorator ใหม่ในอนาคตได้ง่าย

1.3 Decorator Pattern สอดคล้อง Open-Closed Principle อย่างไร

ตอบ Decorator Pattern สอดคล้อง Open-Closed Principle โดยเปิดให้ระบบสามารถเพิ่มความสามารถใหม่ผ่านการสร้าง Decorator ใหม่ โดยไม่จำเป็นต้องแก้ไขคลาสเดิม

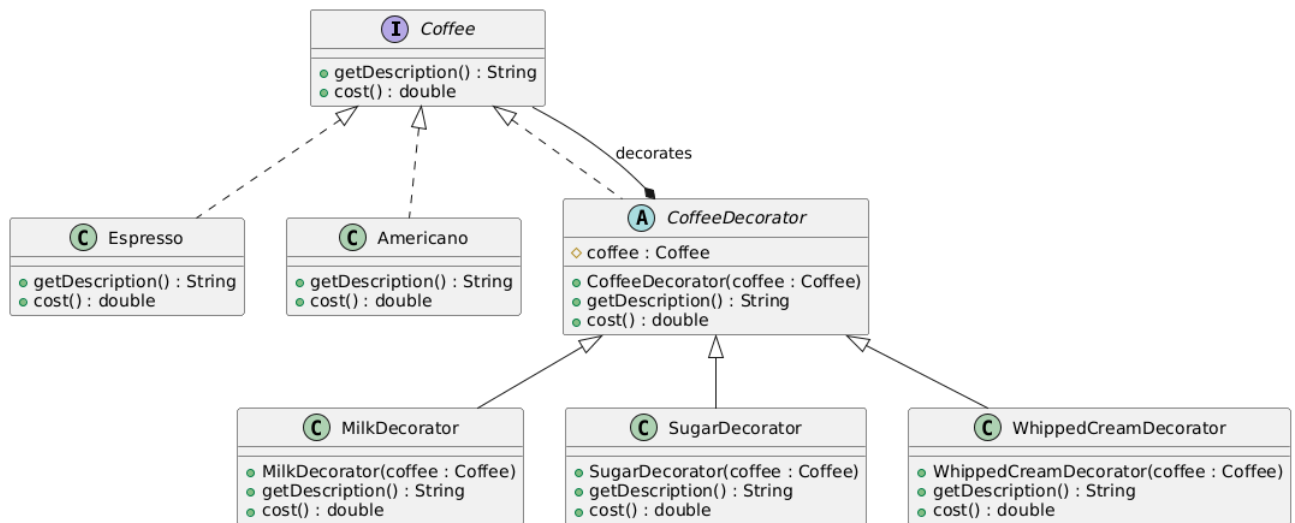
Part 2: UML Design (20 คะแนน)

ออกแบบ UML Class Diagram โดยต้องประกอบด้วย

1. Component ได้แก่ Coffee
2. Concrete Components ได้แก่ Espresso และ Americano
3. Decorator ได้แก่ CoffeeDecorator
4. Concrete Decorators ได้แก่
MilkDecorator, SugarDecorator, WhippedCreamDecorator

โดยแสดง

- Inheritance Relationship
- Association/Composition Relationship
- Method Signatures



Part 3: Implementation (40 คะแนน)

พัฒนาโปรแกรมด้วยภาษา Java โดยใช้ Decorator Pattern

Requirement 1

สร้าง Interface Coffee

```
Java
package src.model;
public interface Coffee {

    String getDescription();
    double cost();
}
```

Requirement 2

สร้าง คลาส Espresso, Americano

```
Java

public class Americano implements Coffee {

    @Override
    public String getDescription(){
        return "Americano";
    }

    @Override
    public double cost(){
        return 2.50;
    }
}

public class Espresso implements Coffee {

    @Override
    public String getDescription(){
        return "Espresso";
    }

    @Override
    public double cost(){
        return 2.00;
    }
}
```

Requirement 3

สร้าง Abstract Decorator ชื่อคลาสชื่อ CoffeeDecorator

```
Java

public abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }

    @Override
    public abstract String getDescription();

    @Override
    public abstract double cost();
}
```

Requirement 4

สร้างคลาสที่ทำหน้าที่เป็น Decorators ได้แก่คลาสต่อไปนี้

6. MilkDecorator

```
Java
public class MilkDecorator extends CoffeeDecorator{
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", Milk";
    }

    @Override
    public double cost() {
        return coffee.cost() + 0.50;
    }
}
```

7. SugarDecorator

```
Java
public class SugarDecorator extends CoffeeDecorator {

    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", Sugar";
    }

    @Override
    public double cost() {
        return coffee.cost() + 0.20;
    }
}
```

8. WhippedCreamDecorator

```
public class WhippedCreamDecorator extends CoffeeDecorator {

    public WhippedCreamDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", Whipped Cream";
    }

    @Override
    public double cost() {
        return coffee.cost() + 0.70;
    }
}
```

Requirement 5

รองรับการเพิ่ม Decorator หลายชั้น

ตัวอย่าง

```
Coffee order = new WhippedCreamDecorator( new SugarDecorator( new MilkDecorator(
    new Americano())));
```

Part 4: Testing and Discussion (15 คะแนน)

ทดสอบอย่างน้อย 3 กรณี

Test Case 1

เลือกกาแฟ เป็น Espresso

```
Test Case 1
Configuration: Espresso
Description: Espresso
Total Cost: $2.00
```

Test Case 2

เลือกกาแฟแบบ Espresso + Milk

```
Test Case 2
Configuration: Espresso + Milk
Description: Espresso, Milk
Total Cost: $2.50
```

Test Case 3

เลือกกาแฟแบบ Americano + Milk + Sugar + Whipped Cream

```
Test Case 3
Configuration: Americano + Milk + Sugar + Whipped Cream
Description: Americano, Milk, Sugar, Whipped Cream
Total Cost: $3.90
```

สำหรับแต่ละกรณีให้แสดง

- Configuration
- Description
- Total Cost

Part 5: Design Reflection (10 คะแนน)

ตอบคำถามต่อไปนี้

5.1 Decorator Pattern แตกต่างจาก Strategy Pattern อย่างไร

ตอบ Decorator Pattern ใช้สำหรับเพิ่มความสามารถหรือพฤติกรรมให้กับ object โดยสามารถเพิ่มได้หลายชั้น ส่วน Strategy Pattern ใช้สำหรับแยกและเลือก algorithm หรือวิธีการทำงานที่แตกต่างกัน ดังนั้น Decorator เน้นการเพิ่มความสามารถให้ object ในขณะที่ Strategy เน้นการเปลี่ยนวิธีการทำงาน

5.2 หากมี Add-on ใหม่เพิ่มขึ้น 20 รายการ ระบบที่ใช้ Decorator Pattern จะมีข้อดีอย่างไร

ตอบ ระบบสามารถเพิ่ม add-on ใหม่โดยสร้าง Decorator ใหม่สำหรับแต่ละ add-on โดยไม่จำเป็นต้องสร้างคลาสสำหรับทุก combination ของ base coffee และ add-on ทำให้ระบบมีความยืดหยุ่นและดูแลรักษาได้ง่ายกว่าแนวทางที่ใช้ Inheritance เพียงอย่างเดียว

5.3 หลักการ "Composition over Inheritance" ปรากฏอยู่ในส่วนใดของงานนี้

ตอบ หลักการนี้ปรากฏใน CoffeeDecorator ซึ่งมีตัวแปรชนิด Coffee สำหรับเก็บ Object ที่ต้องการ Decorate เช่น MilkDecorator สามารถเก็บ Espresso หรือ Americano ไว้ภายใน แล้วเพิ่มความสามารถของตัวเองเข้าไป แทนการสร้างคลาสใหม่สำหรับทุก combination

5.4 จงยกตัวอย่างระบบจริงที่สามารถใช้ Decorator Pattern ได้อีกอย่างน้อย 2 ระบบ

ตอบ 1.ระบบสั่งพิซซ่า ซึ่งสามารถนำ Pizza ไปห่อด้วย Decorator เช่น Cheese, Pepperoni และ Mushroom เพื่อเพิ่มส่วนประกอบได้อย่างยืดหยุ่น

2. ระบบแต่งรถ ซึ่งสามารถใช้ Decorator เพื่อเพิ่มอุปกรณ์เสริม เช่น Turbo, Spoiler หรือ Sound System ให้กับรถพื้นฐานได้

Deliverables

นักศึกษาต้องส่ง

9. Source Code (.java)
10. UML Diagram (.png หรือ .pdf)
11. Test Results พร้อมภาพหน้าจอ
12. รายงาน (2-3 หน้า)

5. Assessment Rubric (100 คะแนน)

หัวข้อ	คะแนน
Problem Analysis	15
UML Design	20
Implementation	40
Testing	15
Design Reflection	10
รวม	100